Evidence, not verdicts: what 479,547 public PX4 flight logs can and cannot tell a drone fleet operator

[First Author][a,*], [Second Author][a]

[a] *[Department, Institution], [City], [Country]*

Abstract

Flight logs are increasingly expected to explain why a small drone crashed. We tested that expectation on the public PX4 Flight Review archive (479,547 logs; 590 crashes labelled by their uploaders as pilot- or aircraft-caused), committing each test and its pass mark to version control before running it. Safety settings did not separate crash logs from well-rated flights ($p = 0.63$). A fixed attribution rule reached a balanced accuracy of 0.47 on 112 held-out crashes, and a cross-validated model stayed between 0.42 and 0.52. Checks that seemed to find the cause named by forum experts mostly fired on the crash itself (13 of 15 tuning faults found with the crash included, 2 of 10 without its last 10 s). The crash flight did not differ from the same drone's earlier flights, but drones that later crashed vibrated more on ordinary flights than matched drones (mean rank 0.67, 95% CI 0.57–0.77, Holm-adjusted $p = 0.016$). Small language-model panels lost to a fixed rule and were easily steered, and matching a written problem description against 90 expert-diagnosed crashes found the expert's cause less often than the base rate (20% vs 32%). We built these findings into an open-source fleet-assurance tool that presents evidence rather than verdicts: an event timeline, a vibration ranking of the fleet, a signed record, and three-dimensional replay and rehearsal of flights. Flight logs support ranking airframes for inspection, not attributing an individual crash.

Keywords: Unmanned aircraft systems, Flight log analysis, Crash investigation, Fault diagnosis, Data leakage, Pre-registration, Fleet assurance

*Corresponding author.

Email address: `[e-mail]` ([First Author])

1. Introduction

Small unmanned aircraft are now flown commercially for survey, inspection, agriculture and delivery, and the rules that govern them place weight on records. In the European Union and the United Kingdom an operator in the specific category must keep records of operations, maintenance and personnel for at least three years [1, 2]; India requires an accident to be reported within 48 hours [3], and the United States within ten days [4]. When something goes wrong, the onboard flight log is the richest record available. Open-source log viewers, commercial fleet platforms and, more recently, machine-learning and language-model prototypes promise to turn that log into an answer: what failed, and whether the pilot or the aircraft was responsible [5, 6].

The question matters beyond engineering curiosity. An investigation report, an insurance claim or a warranty dispute often turns on whether a crash was caused by the person flying or by the aircraft, and an automatic verdict that looks objective can carry weight it has not earned. Fleet operators face a related but different question: not why one flight ended badly, but which of their drones are most likely to fail next and should be inspected first. A log-based method that is good at the second task need not be good at the first, and confusing the two can mislead both investigators and maintainers.

Whether logs can deliver an answer to the first question has, to our knowledge, not been tested on real crashes at scale. Public fault datasets are dominated by simulation or by faults injected on purpose [7, 8, 9], and accident statistics classify outcomes such as loss of control rather than causes visible in a log [10, 11]. Two pitfalls make optimistic results easy to obtain. First, a log that contains a crash also contains its consequences: saturated motors, extreme attitudes and estimator errors. A detector can learn these instead of the cause, a form of data leakage [12, 13]. Second, comparing crash flights with other operators' good flights mixes the flight with the aircraft and the people using it, so a difference between groups need not say anything about the flight that crashed.

This paper asks four questions of the largest public archive of real autopilot logs, PX4 Flight Review [14, 15]:

- Q1 Do configuration settings differ between crash flights and normal flights?
- Q2 Can a log separate crashes caused by the pilot from crashes caused by the aircraft?
- Q3 Does the flight that crashed show warning signs relative to the same drone's history, or are the differences between drones?

Q4 Do expert diagnoses agree with uploader labels, and do automatic checks find the faults experts name?

Because language-model assistants are now proposed for exactly this task, we also report a controlled test of small local models used as a review panel.

The contributions are as follows. (i) We assemble a real-flight crash benchmark from public logs and check its labels against expert diagnoses. (ii) Every confirmatory test was declared, with its threshold and pass mark, before the data it used were analysed, and decisive tests were repeated on held-out data, following the logic of pre-registration [16]. (iii) We show that crash attribution from logs was not achievable with these labels and features, and that apparent diagnostic accuracy can come from the crash itself. (iv) We report one positive and practically useful result: drones that later crash are rougher on their ordinary flights than matched drones, which supports fleet-level inspection ranking but not per-flight prediction. (v) We show how these results translate into the design of an open-source assurance tool, and report engineering checks of its parts, including one negative result (Section 6). Code, declarations, derived features and labels are released.

2. Related work

Fault datasets. Table 1 compares our data with the main public resources. ALFA provides 47 fixed-wing flights with injected engine and control-surface failures [7]. RflyMAD covers 5,629 multicopter fault cases with eleven fault types, of which 497 are real flights and the rest software- or hardware-in-the-loop simulations [8]. UAV-SEAD offers about 1,396 real PX4 logs annotated for state-estimation anomalies [9], used by later anomaly-mining work [17]; a recent testbed quantifies the discrepancies between simulated and real PX4 logs [18]. These resources target anomaly or fault detection under known faults. None labels real crashes by responsibility, and none separates the pre-crash window from the crash.

Fault detection and diagnosis. Surveys of fault detection for UAVs and robots cover model-based and data-driven methods and stress how hard robotic diagnosis is in the field [19, 20]. Causal models built from autopilot documentation have been used to find actual causes of failures in ArduPilot logs [21]. A study of 569 bugs in PX4 and ArduPilot found parameter handling among the root-cause types of drone-specific bugs [22]. Maintenance-oriented relia-

Table 1: Public UAV fault and anomaly data compared with this study.

Resource	Flights	Faults or labels	Real crashes labelled by cause
ALFA [7]	47 fixed-wing flights	Injected engine and control-surface failures	No
RflyMAD [8]	5,629 cases (497 real)	11 injected fault types	No
UAV-SEAD [9]	about 1,396 real PX4 logs	State-estimation anomalies (4 classes)	No
This study	479,547-log index; 590 crash-rated logs; 148 expert-diagnosed threads	Uploader responsibility labels; expert causes	Yes (uploader; partly expert-checked)

bility analysis of UAVs models the passage from partial degradation to failure [23], the view our between-drone result supports.

Accident analysis. Analyses of civil drone occurrences find equipment problems and loss of control prominent [10]. Human-factors classifications of remotely piloted aircraft mishaps attribute a large share of causal factors to people [24, 25]. Such classifications depend on the rater, which is one reason we check uploader labels against expert diagnoses.

Language models for diagnosis. Language-model agents have been applied to simulation testing and log analytics for small UAVs [5], and learned models to classifying problems in drone log messages for forensics [6]. Multi-agent debate has been proposed to improve the factuality of model outputs [26], while indirect prompt injection shows that text inside data can steer model behaviour [27]. We test both properties on our task.

Methodology. Leakage, where information unavailable at prediction time enters training or evaluation, is a known source of irreproducible machine-learning results [12, 13]; label noise likewise biases evaluation [28]. Matching reduces confounding in observational comparisons [29], and pre-registration

separates confirmatory from exploratory analysis [16]. We apply these ideas to drone logs.

Tools for operators. Retrieving similar past cases and reusing their solutions is the basis of case-based reasoning [30]; ranking text by word overlap with BM25 is a standard, deterministic retrieval method [31]. A hand-held miniature of the environment lets a person grasp and inspect a spatial scene at a glance [32], the idea behind the tabletop view in Section 6. We use these as engineering choices and test them only as far as stated.

3. Data

3.1. Archive and labels

PX4 Flight Review is a public service to which PX4 users upload onboard logs (ULog format) for plotting and sharing [15]. We downloaded its public index on 20 October 2026: 479,547 logs (32,793,172 bytes, SHA-256 `1bbb01c3…a92d6`), of which 442,929 came from real hardware rather than simulation, from 38,320 distinct vehicle identifiers. Uploaders may rate a log. A total of 590 logs carry a crash rating, 402 attributing the crash to software or hardware ("aircraft-caused") and 188 to the pilot ("pilot-caused"). Logs rated "good" or "great" served as normal flights. Logs were parsed with pyulog. We store derived features and log identifiers, not the logs.

A ULog file contains the parameters in force when logging started, any parameter changes during logging, timestamped messages published by the autopilot's modules, and logged text messages with a severity level. The analyses below use the vehicle status (arming state, navigation mode, failsafe flags), attitude and body-rate topics, rate setpoints, inertial-sensor status, actuator outputs, battery status, GPS position and the land detector. The vehicle identifier in the index is derived from the flight controller's hardware serial number; it identifies a flight controller, which in practice usually means one airframe, though a controller can be moved between frames.

Crash-rated logs differ from well-rated logs in composition (Table 2): they are shorter (median 54 s against 388 s, partly because a crash ends the log) and spread over more firmware generations. All between-group comparisons below therefore use either matching or within-drone designs, and we do not pool across firmware generations where measurements differ.

Table 2: Composition of crash-rated and well-rated logs from real hardware in the archive.

	Crash-rated	Rated good or great
Logs	520	21,112
Median duration	54 s	388 s
Most frequent years	2018 (91), 2025 (89), 2019 (87), 2017 (57)	2018 (10,571), 2017 (5,748), 2019 (2,910)
Most frequent PX4 versions	1.8 (70), 1.9 (66), 1.15 (65), 1.11 (65)	1.7 (6,299), 1.8 (5,785), 1.6 (4,265)
Vehicle types	quadrotor 320, fixed wing 71, hexarotor 38	quadrotor 19,890, fixed wing 620, hexarotor 291

3.2. Samples

Figure 1 and Table 3 show how each analysis sample was drawn. Unless stated, crash samples used real hardware, PX4 1.10 or newer and at least 5 s of flight; normal flights were matched to crashes on vehicle type and drawn from vehicles without any crash rating. For the within-drone test every crash with a vehicle identifier and at least three earlier non-crash flights of 30 s or more was eligible (101 crashes; up to the five most recent earlier flights each). For the between-drone test each crashed drone was matched with up to two drones without any crash-rated log that shared the flight-controller hardware, the PX4 major.minor version and the vehicle type (66 matched sets, 124 control drones, three randomly chosen flights each, seed 7). Control drones were sampled without replacement, so no control served two cases. The three exposure flights of a crashed drone were its most recent non-crash flights dated on or before the crash; flights on the crash day were included in the declared test because the archive does not give their order within a day, and removed in a sensitivity analysis.

3.3. Expert diagnoses

We searched the PX4 discussion forum for threads that link a Flight Review log and mention a crash (443 threads, 410 with a log link). For the 161 threads with a reply from PX4 staff, the development team or a member at the forum's highest trust levels (148 of them describing a real crash), we recorded the expert's conclusion (aircraft, pilot, both, unclear, no diagnosis), the cause named, who gave it, whether the original poster confirmed it, and the sentence stating the diagnosis. Recording was AI-assisted and reviewed

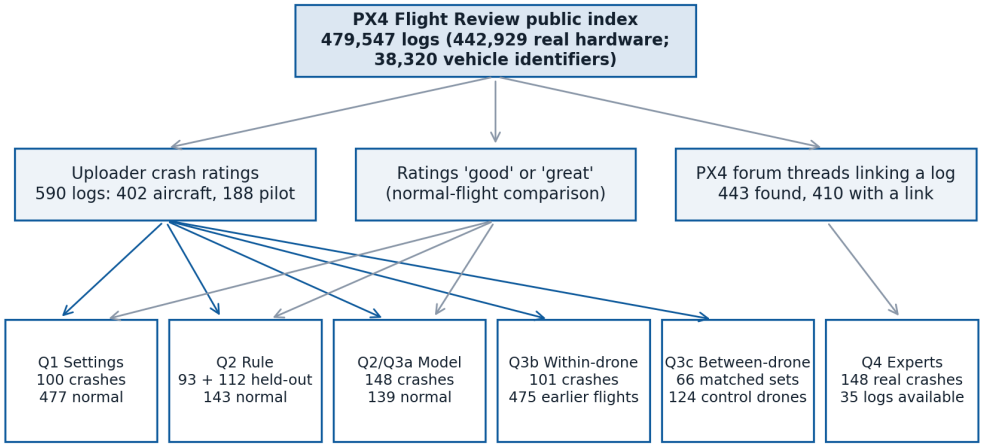


Figure 1: From the public index to each analysis. Counts are logs unless stated.

by an author; 123 of 124 recorded sentences were found verbatim in their thread by script. The searches combined the Flight Review host names with the words crash, crashed, flyaway, fell, flipped and lost control, and every returned thread was retrieved through the forum's public interface. A reply counted as expert when its author was marked as staff, belonged to the development team group or held one of the forum's two highest trust levels. Only public posts were used; usernames and post text are not redistributed.

4. Methods

4.1. Declaration protocol

For each test, the selection, features, statistic, threshold and pass mark were committed to a public version-control repository before the data it used were downloaded or analysed. Commit identifiers are listed in the repository's paper notes. One amendment was made before any analysis: the firmware filter of the within-drone test was removed because it left only 12 cases, and a fallback for older firmware was added. One coding error was corrected after a first run (logs whose CPU-load record lacked the load field were being skipped); no threshold changed. The robustness checks in Section 5.5 were added afterwards and are labelled exploratory.

Table 3: Analysis samples.

Sample	Crashes	Comparison	Question
Settings	100	477 normal (settings section only)	Q1
Development crashes	93 analysed	–	Q2 rule
Held-out crashes	112 analysed	143 normal (full logs)	Q2 replication
Leak-curve set	148 at $m = 0$	139 normal	Q2 model, Q3a
Within-drone	101 selected (61 and 46 analysed)	475 earlier flights of the same drones	Q3b
Between-drone	66 matched sets (60 and 47 analysed)	124 matched drones, 372 flights	Q3c
Forum threads	148 real crashes	35 logs still downloadable	Q4

4.2. Crash moment and analysis window

For rotary-wing aircraft the crash moment t_c is the first time after take-off at which the tilt $\theta = \arccos\left(1 - 2(q_x^2 + q_y^2)\right)$, computed from the attitude quaternion, exceeds 60°; for fixed-wing aircraft, and when no such tilt occurs, it is the end of the armed period. Normal flights use the end of the armed period. Take-off is the first sample after arming at which the land detector reports the vehicle airborne. An analysis with cut-off m uses the window from take-off to $t_c - m$ and is skipped if the window is shorter than 10 s.

4.3. Features

Table 4 defines the features. Vibration is not measured the same way across firmware generations: newer logs report an accelerometer vibration metric, older ones an estimator vibration estimate. Comparisons are therefore only made between flights measured with the same source, and the source is recorded with every value. The rate-tracking error falls back to the attitude message's body rates when the angular-velocity topic is absent.

4.4. Tests

Q1 settings. For logs that carry the firmware's record of default values, a protection was counted as disabled when its value differed from the default

Table 4: Features computed on each analysis window.

Feature	Definition
Rate-tracking error	RMS of commanded minus measured body rate, roll and pitch (rad s^{-1}; rotary wing)
Tilt (p95)	95th percentile of tilt angle (deg)
Vibration (p95)	95th percentile of the accelerometer vibration metric, or of the estimator vibration estimate on older firmware
Clipping rate	accelerometer clipping events per second
Motor saturation	share of time with any motor output at 95% or more (rotary wing)
Battery minimum	lowest reported remaining charge (%)
Voltage sag	(start voltage − minimum voltage) / start voltage
Poor GPS	share of time with horizontal accuracy above 5 m or fewer than 6 satellites
Failsafe	1 if a failsafe was active at any time
Manual share	share of time in a pilot-flown mode (manual, altitude, position, acro, stabilised)
Stick deflection (p95)	95th percentile of the larger roll or pitch stick deflection

and switched it off (for example a failsafe action of "none"). Crash and normal logs were compared with Fisher's exact test.

Q2 attribution. The declared rule labelled a crash "aircraft" if any of eight signs fired before t_c (a critical battery warning or remaining charge below 10%; any motor at 95% or more for at least 1 s in the last 10 s; more than 50 clipping events; poor GPS in a GPS-dependent mode; a failure-detector flag; an error-level message; CPU load above 95%; an active failsafe), otherwise "pilot" if the mode at t_c was pilot-flown, otherwise "unknown". The pass mark was a balanced accuracy of at least 0.70 on decided cases, answering at least 60% of crashes and being wrong on at most 20%. A model-based check used L2-regularised logistic regression on all features ($\lambda = 1$, standardised inputs, median imputation within training folds), evaluated by stratified five-fold cross-validation repeated five times and summarised by the ROC AUC [33].

The signs were chosen before the analysis to cover common failure modes of PX4 multirotors: power, propulsion, vibration, navigation, software and failsafes. The pilot rule was deliberately simple: if no aircraft sign fired and

the vehicle was in a mode in which the pilot controls attitude or position directly, the crash was attributed to the pilot. The rule was fixed before it was run on any crash log; only the list of message types available in the logs had been inspected.

Q3a crash window. The same model separated crash from normal flights at cut-offs $m \in \{0, 2, 5, 10, 20, 30\}$ s. We declared that the AUC would fall by at least 0.10 between $m = 0$ and $m = 10$ s if crash consequences drove it.

Q3b and Q3c rank tests. For a crashed drone i with value x_{i0} and comparison values $x_{i1}, \ldots, x_{in_i}$, the scaled rank is

$$r_i = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbf{1}\left[x_{ij} < x_{i0}\right], \tag{1}$$

which is 0 when the crashed drone has the lowest value and 1 when it has the highest; ties count against the hypothesis. A rank statistic was chosen for three reasons: the two vibration sources have different units and scales, so only orderings are comparable across drones; the number of comparisons differs between drones, which the exact null handles directly; and the distributions are skewed, with a few very rough flights, which would dominate a mean difference. In the within-drone test (Q3b) x_{i0} is the crash flight, cut 10 s before t_c, and the comparisons are the same drone's earlier flights, each cut 10 s before the end of its armed period. In the between-drone test (Q3c) x_{i0} is the median over the crashed drone's three most recent pre-crash flights and the comparisons are the medians of the matched controls. Under no difference each r_i is uniform on $\{0, 1/n_i, \ldots, 1\}$ with mean 0.5, and $\bar{r}$ estimates the probability that the crashed drone exceeds a comparison, the quantity estimated by the Mann–Whitney statistic [34]. One-sided p-values come from 100,000 draws of this exact null; 95% confidence intervals from 10,000 bootstrap resamples of drones [35]. Each test had two primary features (vibration, rate-tracking error) at $\alpha = 0.025$.

Q4 expert diagnoses. Expert conclusions were compared with uploader labels. Fixed checks for the fault the expert named (tuning, vibration, GPS or compass, motor, battery) were declared useful if they fired on at least 50% of the logs with that diagnosis and at most 10% of normal flights, first with the full window and then with $m = 10$ s.

Language-model panel. The panel tested a design that has been proposed for advisory diagnosis: several reviewers, each restricted to one slice of evidence (navigation, pilot inputs, GPS, battery, settings), vote on a fixed list of six causes and must cite numbered facts that code extracted from the log. Citations to facts outside a reviewer's slice or to non-existent facts were discarded by code. In the second round each reviewer saw only the vote counts of the first round, not the other reviewers' text, to limit persuasion. A cause was reported only if it had at least one supporting vote, a net score of at least one and a lead of at least one over the runner-up; otherwise the case was referred to a person. Sixty generated cases with a known cause (clear, subtle and no-cause cases, some with distractors) were presented to five reviewers; a single agent shown all facts and the fixed rule served as comparators. Four local models of 3 to 4 billion parameters ran at temperature 0. Robustness was measured by replacing one reviewer with an adversary arguing for a chosen wrong cause. Separately, eight hostile strings were planted in five uploader-controlled fields to test whether text could reach a prompt.

Multiplicity. Seven confirmatory tests produce p-values. Holm's procedure [36] is reported over all seven in addition to each test's declared α.

5. Results

5.1. Configuration settings (Q1)

Of 57 crash logs and 113 normal logs carrying a firmware-default record, 29 (51%) and 52 (46%) had at least one protection disabled relative to the default (Fisher $p = 0.63$). Pilot-caused crashes had a protection disabled more often than aircraft-caused ones (14 of 20 against 15 of 37), a contrast too small to test. None of 100 crash logs changed a safety-critical or flight-limit setting during the flight.

5.2. Attributing a crash to the pilot or the aircraft (Q2)

The fixed rule reached a balanced accuracy of 0.57 on 93 development crashes (35% wrong) and 0.47 on 112 held-out crashes (47% wrong), against the declared 0.70, and labelled 39% of 143 normal flights as aircraft faults (Table 5). The signs fired at similar rates whichever party the uploader blamed (Fig. 2): motor saturation in 23% of aircraft-caused and 29% of pilot-caused crashes (1% of normal flights), battery warnings in 15% and 18% (3%). Error messages and failsafes were no more frequent in crashes

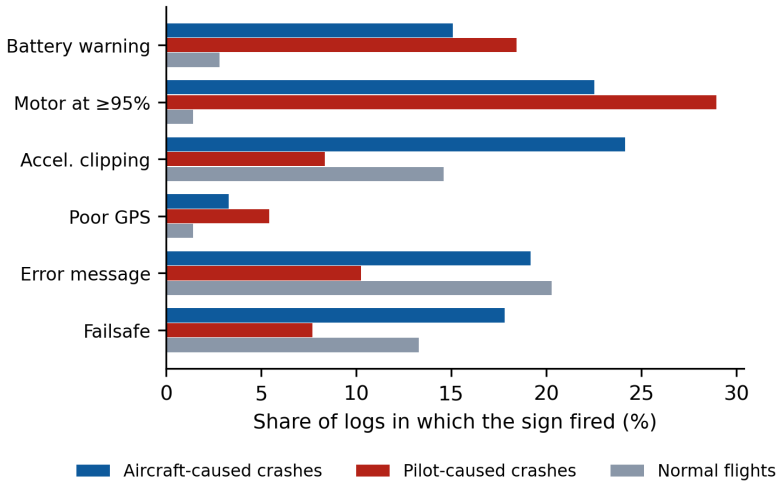


Figure 2: How often each aircraft-fault sign fired in held-out crashes, by uploader label, and in normal flights.

than in normal flights. Two signals noticed after the development run did not replicate on held-out crashes: accelerometer clipping (24% against 8%, $p = 0.045$) and manual mode at the crash (18% against 10%, $p = 0.165$). The cross-validated model stayed between 0.42 and 0.52 at every cut-off (Fig. 3).

5.3. The crash window (Q3a)

Crash flights were separated from normal flights with an AUC of 0.88 (SD 0.01) using the whole flight and 0.86 with the last 30 s removed (Fig. 3); the declared drop of 0.10 by 10 s was not observed (0.01). Leakage was nevertheless present for individual checks. The rate-tracking feature alone fell from an AUC of 0.81 to 0.69 as the crash was excluded, while vibration stayed between 0.74 and 0.77. A fixed tuning check that found 13 of 15 expert-diagnosed tuning faults with the crash included found 2 of 10 once the last 10 s were removed (Section 5.6). Model-level separation therefore does not depend on the crash window; it reflects differences that are present throughout the flight. Table 6 gives the single-feature AUCs. Values below 0.5 indicate features that are lower in crash flights: crash flights reached lower tilt and smaller stick deflections over the whole flight than normal flights, consistent with shorter and less aggressive flights that end abruptly.

Table 5: Confirmatory tests, their declared pass marks and outcomes. Holm adjustment over the seven p-valued tests.

Test (declared criterion)	Result	Holm p	Outcome
Protections disabled, crash vs normal	51% vs 46%, $p = 0.63$	1.00	No difference
Attribution rule, development (BA ≥ 0.70)	BA 0.57, 35% wrong	–	Failed
Attribution rule, held-out	BA 0.47, 47% wrong; 39% of normal flagged	–	Failed
Clipping, aircraft- vs pilot-caused ($p < 0.025$)	24% vs 8%, $p = 0.045$	0.27	Not supported
Manual mode, pilot- vs aircraft-caused ($p < 0.025$)	18% vs 10%, $p = 0.165$	0.73	Not supported
AUC drop from 0 to 10 s (≥ 0.10)	0.88 to 0.87	–	Not supported
Within-drone vibration ($p < 0.025$)	$\bar{r} = 0.48$, $p = 0.67$	1.00	Not supported
Within-drone tracking ($p < 0.025$)	$\bar{r} = 0.50$, $p = 0.47$	1.00	Not supported
Between-drone vibration ($p < 0.025$)	$\bar{r} = 0.67$, $p = 0.0023$	0.016	Supported
Between-drone tracking ($p < 0.025$)	$\bar{r} = 0.57$, $p = 0.15$	0.73	Not supported
Expert-fault checks, $m = 10$ s ($\geq 50\%$, $\leq 10\%$)	best: vibration 3/3 vs 11%	–	None met

5.4. Within the same drone and between drones (Q3b, Q3c)

Relative to the same drone's earlier flights, the crash flight was neither more vibrating ($\bar{r} = 0.48$, 95% CI 0.38–0.58, $p = 0.67$, 61 drones) nor worse at tracking its commands ($\bar{r} = 0.50$, 0.38–0.63, $p = 0.47$, 46 drones; Fig. 4). Relative to matched drones without a recorded crash, drones that later crashed vibrated more on their ordinary pre-crash flights ($\bar{r} = 0.67$, 0.57–0.77, $p = 0.0023$, 60 sets; Holm-adjusted $p = 0.016$), while the tracking error did not differ significantly ($\bar{r} = 0.57$, 0.44–0.70, $p = 0.15$, 47 sets). The vibration difference was clearest for aircraft-caused crashes ($\bar{r} = 0.67$, $p = 0.005$, 46 sets) and similar in size but not significant for pilot-caused

Table 6: Single-feature AUC for separating crash from normal flights at three cut-offs (no fitting; 0.5 is chance; values below 0.5 mean lower in crash flights).

Feature	$m = 0\,\mathrm{s}$	$m = 10\,\mathrm{s}$	$m = 30\,\mathrm{s}$
Rate-tracking error	0.81	0.72	0.69
Vibration (p95)	0.74	0.72	0.77
Manual share	0.67	0.68	0.69
Motor saturation	0.67	0.66	0.65
Clipping rate	0.59	0.54	0.55
Voltage sag	0.55	0.53	0.48
Poor GPS	0.53	0.55	0.54
Failsafe	0.50	0.47	0.47
Battery minimum	0.42	0.40	0.38
Tilt (p95)	0.41	0.36	0.33
Stick deflection (p95)	0.32	0.33	0.35

ones ($\bar{r} = 0.64$, $p = 0.16$, 14 sets). The crash-versus-normal separation of the previous subsection is therefore a property of the drone rather than a build-up within the flight that crashed.

5.5. Robustness of the between-drone result (exploratory)

The vibration result held when controls were restricted to the same airframe as the case ($\bar{r} = 0.69$, $p = 0.008$, 39 sets), when case flights from the crash day were removed ($\bar{r} = 0.76$, $p = 0.003$, 25 sets) and after adjusting for flight duration ($\bar{r} = 0.69$, $p = 0.0005$, 57 sets; Fig. 5). Duration was barely correlated with vibration (Spearman $\rho = 0.05$ and -0.04 for the two measurement sources), although case flights were longer than control flights (median 289 s against 173 s). The effect was carried by the older estimator vibration estimate ($\bar{r} = 0.68$, $p = 0.005$, 48 sets); with the newer accelerometer metric alone the direction was the same but the sample was too small to decide ($\bar{r} = 0.62$, $p = 0.19$, 12 sets). The within-drone null did not depend on the crash-moment threshold ($\bar{r} = 0.46$ at 45°, 0.50 at 75°). As a screening signal the effect is moderate: the crashed drone was the roughest of its matched set in 55% of sets, against 41% expected without a difference.

5.6. Expert diagnoses (Q4)

Of 148 forum threads describing a real crash, the expert attributed it to the aircraft in 88 (59%), to the pilot in 9 (6%), to both in 2, and was

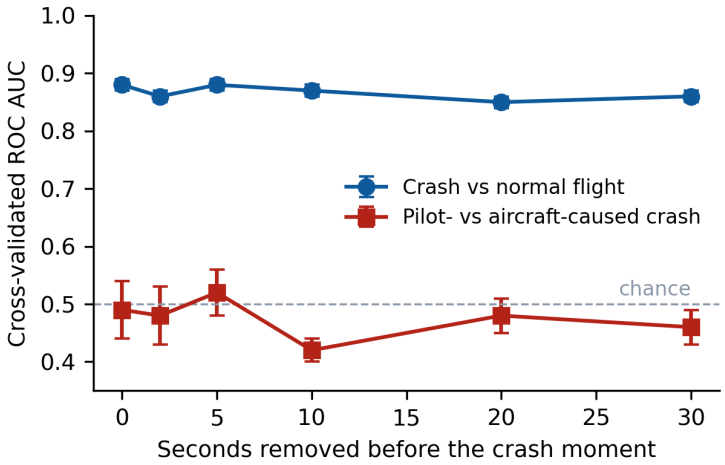


Figure 3: Cross-validated AUC as the end of the analysis window moves away from the crash moment. Bars: standard deviation over five repeats of five-fold cross-validation.

unclear or gave no diagnosis in 49 (33%). Tuning or configuration was the cause named most often (29 of 88; 15 of the 47 confirmed by the poster), followed by GPS, compass or estimator problems (16) and motors, ESCs or propellers (14) (Fig. 6). Of 14 threads whose log carried an aircraft crash rating, the expert agreed in 9 and was unclear in 5; none attributed the crash to the pilot. Only 35 of the 70 logs behind aircraft diagnoses could still be downloaded. Table 7 shows the fault checks with and without the crash window. With the window ending 10 s before the crash, no check met the declared criterion. The tuning check illustrates the leakage described in Section 5.3: its apparent sensitivity fell from 87% to 20% when the last 10 s were removed, while its rate on normal flights fell only from 13% to 5%.

5.7. Language-model review panels

The fixed rule was right in 45 of 60 generated cases (75%, 95% Wilson interval 63–84%) and wrong in none (0%, 95% interval 0–6%); it referred the other 15 to a person. "Right" includes leaving the 10 no-cause cases open, so the rule named the true cause in 35 of the 50 cases that had one. No model's panel matched it (Table 8): wrong answers ranged from 6 to 38 of 60 (10–63%). A single compromised reviewer changed the panel's answer in 45 to 56 of 60 cases. None of 40 planted hostile strings reached a model prompt, because the pipeline passed only numbers extracted by code.

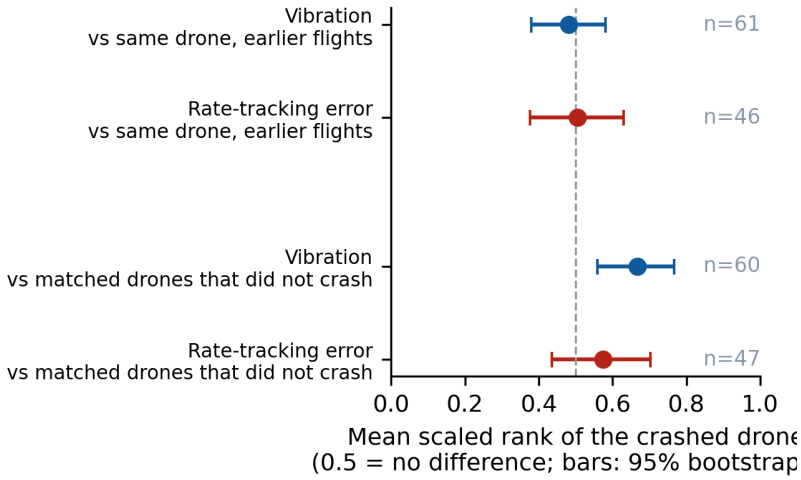


Figure 4: Mean scaled rank of the crashed drone against its own earlier flights (top) and against matched drones without a recorded crash (bottom). 0.5 indicates no difference.

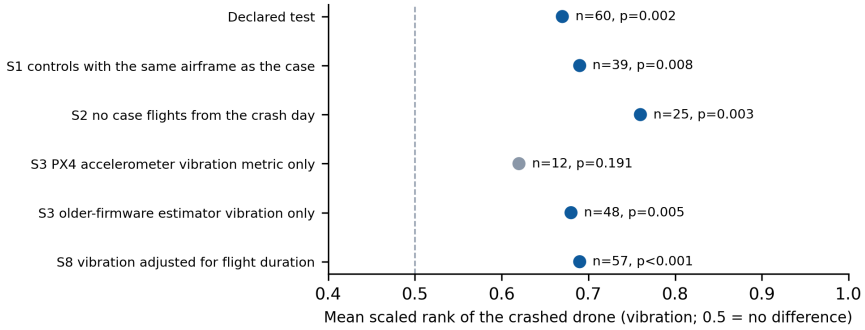


Figure 5: Between-drone vibration result under alternative specifications (exploratory). Blue: $p < 0.025$.

Table 7: Fault checks on logs whose expert named the matching fault, with the full window and with the last 10 s before the crash removed. Declared criterion: at least 50% of diagnosed logs and at most 10% of normal flights.

Expert-named fault	Full window: diagnosed	Full window: normal	$m = 10$ s: diagnosed	$m = 10$ s: normal
Tuning or configuration	13/15 (87%)	18/142 (13%)	2/10 (20%)	7/130 (5%)
Vibration or mounting	3/3	7/48 (15%)	3/3	4/38 (11%)
GPS, compass or estimator	0/4	10/143 (7%)	0/3	5/130 (4%)
Motor, ESC or propeller	2/7 (29%)	8/143 (6%)	1/4 (25%)	5/130 (4%)
Battery or power	0/4	4/143 (3%)	0/2	1/130 (1%)

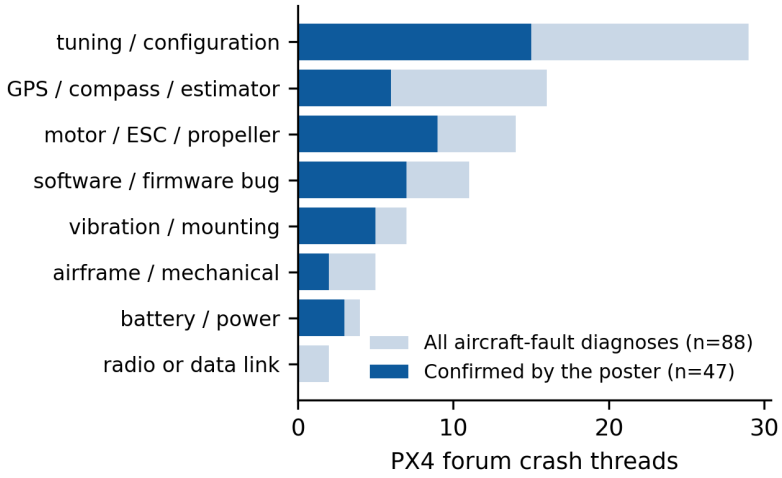


Figure 6: Causes named by experts in PX4 forum threads that attributed a crash to the aircraft.

Table 8: Language-model review panels on 60 generated cases: counts, with 95% Wilson intervals for the share of all 60 cases. "Right" counts naming the true cause or correctly leaving a no-cause case open.

Method	Single agent: right / wrong	Panel: right	Panel: wrong	Panel answer changed by one adversary
Fixed rule (no model)	45 (63–84%) / 0 (0–6%)	–	–	–
gemma3:4b	10 / 0	3 (2–14%)	36 (47–71%)	49 of 60
phi4-mini	10 / 0	20 (23–46%)	19 (21–44%)	56 of 60
qwen2.5:3b	9 / 44	10 (9–28%)	6 (5–20%)	54 of 60
qwen3:4b	0 / 41	17 (19–41%)	38 (51–74%)	45 of 60

6. Design implications: an assurance tool built on the findings

The authors develop AeroFleet, an open-source, ground-side assurance tool for PX4 and ArduPilot fleets. It reads the files an operator already has (parameter files, mission files and flight logs), contacts no aircraft and calls no language model. Table 9 links each result above to a design decision and to the check that was made of it. The injection and leave-one-out checks were declared before they were run, like the tests above; the others are software tests. All are engineering validation; none evaluates the tool as a safety intervention.

Evidence, not verdicts. A flight review reads an ArduPilot or PX4 log and the approved mission file and lists, in time order, mode changes (with a pilot taking over from an automatic mode flagged), settings changed in flight, battery and voltage warnings, poor GPS and deviations from the plan. It reports the quality of that evidence (sample rate, satellite count and dilution of precision, height reference, clock source) so that a reader can judge how far to trust it, and leaves the conclusion to a person. Given the attribution results, a tool that issued a pilot-or-aircraft verdict would claim more than the log supports.

Ranking drones, not predicting flights. Each reviewed flight's vibration is stored with its measurement source; drones are ranked against others measured the same way, and those above their group's upper quartile are flagged for mechanical inspection. In the archive, this kind of ranking identified the drone that later crashed as the roughest of its matched set in 55% of sets, against 41% by chance, so the flag is a prompt to inspect, not a warning that a crash is coming.

A record that can be checked. Because the useful evidence accumulates per aircraft, results are appended to a hash-chained record in which each entry is signed and carries the SHA-256 of the exact files it came from; each new head is also written to a separate file intended to be copied elsewhere. A standalone verifier checks an exported record; in tests it detected an edited field, a deleted entry, a reordering, and a chain rebuilt with a new key (the last only against a pinned key fingerprint or the separately kept heads, which is the stated limit of the design).

Table 9: From findings to design decisions in the assurance tool, and how each part was checked.

Finding	Design decision	Check
Logs did not attribute crashes (Q2); checks fired on the crash (Q4)	A flight review lists the log's events in time order and states the quality of its evidence; it names no cause	Unit and API tests
Roughness marked drones, not flights (Q3b, Q3c)	Drones ranked by vibration within a fleet, measured the same way; the upper quartile flagged for inspection	Unit tests; no prospective evaluation yet
Useful evidence is longitudinal	Every upload, approval and review appended to a hash-chained, Ed25519-signed record with SHA-256 fingerprints of the files received	Tamper tests: edited, deleted, reordered and re-signed records detected
A plan proves nothing if something else was flown	Planned-versus-flown check: deviations beyond a fleet's corridor and height tolerance	Known deviations injected into 40 real flights (Section 6.1)
Expert diagnoses as precedents (Q4)	Description search over 161 expert-labelled threads; checklist in order of expert frequency	Leave-one-out test on 90 cases (Section 6.2)
Separation and deviations are three-dimensional	Tabletop 3D replay of a reviewed flight and rehearsal of planned missions, on screen or in a headset	Functional checks only; no user study

6.1. Planned versus flown

The check aligns the flown track with the plan's legs in flying order over the whole flight and reports stretches beyond the fleet's sideways and height tolerances (defaults 30 m and 15 m for at least 3 s). To test it, plans were built from 40 real multirotor flights in the archive (a waypoint every 10 s of track), so that the unmodified flight conformed by construction, and a known deviation was injected for 30 s. Sideways shifts of 35 m or more were found in 90–98% of flights and upward shifts of 20 m or more in 92–98%, while shifts below the tolerances were mostly not reported, as specified. The first version of the check, which matched each position to the nearest leg anywhere, raised false alarms on 16 of 40 unmodified flights and detected only half of the 35 m shifts; it was replaced. The current version raised false alarms on 9 of 40 (22%, 95% CI 12–38%), all on fast flights cutting the corners of long legs built from the track; with a waypoint every 5 s (a post-hoc check) there were none. A real mission has waypoints at its turns, but the false-alarm rate on real plans has not been measured, so a reported deviation is a reason to look at the replay rather than a finding.

6.2. Searching past diagnoses

An operator can describe a problem in words. The description is matched (BM25, no model) against the expert-labelled forum threads, represented by their titles and up to 30 descriptive terms from the opening post, and against a short list of symptom words for each cause. After a first release we declared and ran a test, holding out each of the 90 threads in which the expert blamed the aircraft and named a cause and querying with the rest. The cause the words matched best was the expert's cause in 17 of 90 cases with the title alone (19%) and 18 of 90 with the title and opening-post terms (20%, 95% CI 13–29%), below the 29 of 90 (32%) of always answering the most frequent cause; among the first three it was 53% against 66%. The declared criterion (10 points above the base rate) was not met. A written description, like the log, did not reveal the cause. The tool therefore orders its checklist by how often experts found each cause, marks the causes the description mentions, and offers the most similar threads only as reading, with the measured accuracy shown beside the answer.

6.3. Three-dimensional replay and rehearsal

Deviations, heights and separation between drones are spatial, and a two-dimensional plot hides height. The tool therefore draws flights on a virtual

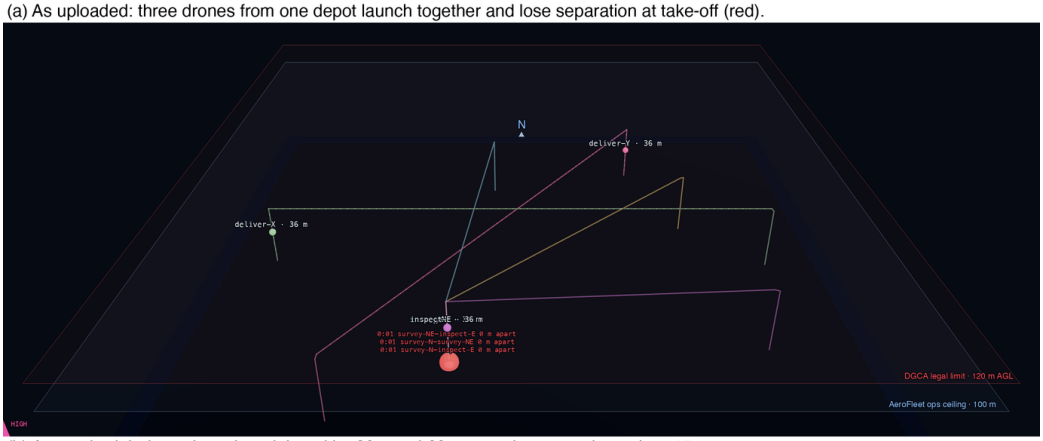
tabletop, a world in miniature [32], that can be viewed in a browser, in immersive virtual reality or, on a passthrough headset, in mixed reality on a real desk (WebXR). A *flight replay* shows a reviewed flight: the approved plan dashed, the flown track green within tolerance and red during a deviation, the log's events pinned where they happened, and a playhead that can be scrubbed or played at ten times real speed. A *mission rehearsal* shows planned missions before anyone flies: the deconfliction check turns each mission file into a position-over-time trajectory on a common clock (straight legs at stated climb, cruise and descent speeds), finds every pair closer than the separation distance and resolves conflicts by a launch delay or, where allowed, a different cruise height. The rehearsal flies all missions on that clock, as uploaded and as rescheduled, with each loss of separation drawn where and when it occurs (Fig. 7). It is a kinematic rehearsal of the plan, not a simulation of the aircraft: there is no wind, no vehicle dynamics and no sensor error, so it shows what the plan implies if it is flown as written, which is the assumption the planned-versus-flown check then tests after the flight. The replay and rehearsal have been checked functionally on desktop; a paired standalone headset application compiles but has not been run on a headset, and no user study has been carried out, so we make no claim that the three-dimensional view improves decisions.

7. Discussion

Attribution. A declared rule, its held-out replication and a cross-validated model all found no usable separation between pilot- and aircraft-caused crashes. Uploader labels are noisy, but the expert threads suggest they are not random: where an expert reached a verdict on an aircraft-rated crash, it agreed. A more likely explanation is that a pilot error and an aircraft fault leave similar traces in the signals a log records, which is also suggested by how often experts themselves remained unclear. Tools that issue automatic responsibility verdicts from logs should be evaluated against independent labels before use in investigations or claims.

Leakage. The model-level leakage we expected was absent, yet single checks showed it clearly: a tuning check looked accurate only because the crash was in its window. Evaluations of log-based diagnosis should report results with the crash window removed, and should compare against the same aircraft's history before claiming that a signal anticipates a crash.

(a) As uploaded: three drones from one depot launch together and lose separation at take-off (red).



(b) As rescheduled: two launches delayed by 30 s and 60 s; no pair comes closer than 15 m.

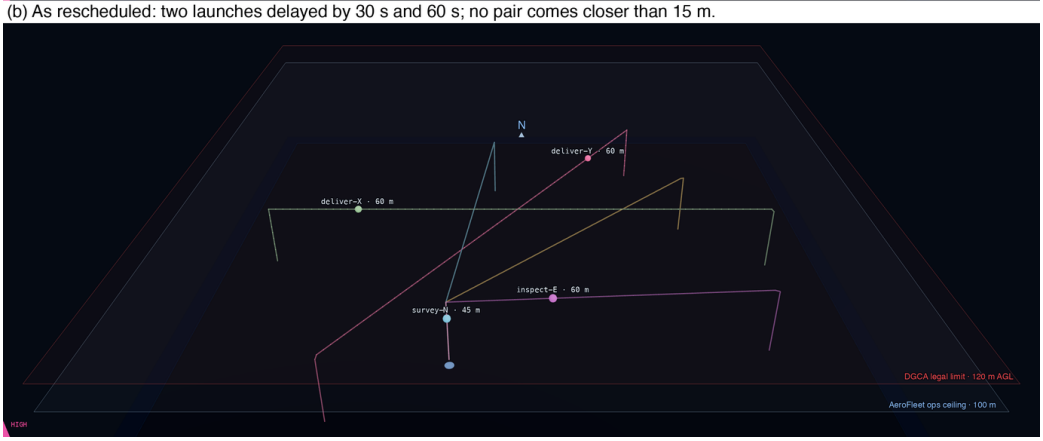


Figure 7: Mission rehearsal in the tool's tabletop view (desktop rendering; the same scene opens in a WebXR headset). Five planned missions over a 2 km table, heights exaggerated four times, each mission in its own colour. (a) As uploaded, 12 s after launch: three missions leaving one depot together occupy the same point (overlapping labels) and the red sphere marks their loss of separation. (b) The schedule produced by the deconfliction check, 45 s after the first launch: two launches are delayed and no pair comes within 15 m. Straight legs at planned speeds, no wind.

Drone-level risk. The positive result concerns airframes, not flights. Drones that later crashed vibrated more than comparable drones on ordinary flights, and expert diagnoses most often named tuning, configuration and propulsion problems, which raise vibration. This is consistent with a maintenance view of UAV reliability in which degraded condition precedes failure [23]. It supports ranking a fleet's drones by vibration measured the same way and inspecting the roughest; it does not support a per-flight crash warning, and the screening gain is moderate.

From findings to a tool. Section 6 turns these results into design decisions for an operator's tool. The common thread is that the tool shows evidence and ranks drones, and does not name a cause.

Language models. On this task small local models added errors and an attack surface rather than accuracy. A debate-style panel, related to multi-agent debate [26], did not help at this model size, and the adversarial result echoes concerns about steering through injected content [27]. Larger models were not tested.

Relation to prior datasets. Benchmarks built from injected faults [7, 8] answer whether a known fault can be detected; anomaly datasets from real logs [9] answer whether unusual segments can be found. Our results concern a different question, attribution of a real crash, and suggest that performance on the first two does not transfer to it: signals that mark a fault also mark the crash it causes and the pilot's reaction to it. Datasets that pair real crashes with independently verified causes would be needed to make progress on attribution, and our expert-diagnosis labels are a small first step.

Records and regulation. Record-keeping rules ask operators to keep logs, maintenance records and occurrence reports [1, 2]. On our evidence, such records are most useful when they are kept per aircraft over time, so that an airframe's condition can be compared with its own history and with comparable airframes, rather than examined only after an accident.

Practical guidance. For operators and investigators we suggest: (i) treat automatic pilot-or-aircraft verdicts from logs with caution; (ii) use logs to assemble a time-ordered record of events for a human investigator; (iii) track vibration per drone across flights and inspect drones that are rough relative to comparable airframes; (iv) when evaluating any log-based diagnostic, remove the crash window and compare against the same drone's history.

Future work. Three extensions would strengthen the evidence: repeating the between-drone analysis on ArduPilot fleets and on newer PX4 firmware, where the accelerometer vibration metric is available; a prospective design in which vibration rankings are recorded before crashes occur; and a larger set of crashes with causes verified by independent investigators.

8. Threats to validity

Labels. Ratings are the uploaders' own and normal flights are self-rated. Controls without a crash-rated log may have crashed without a rating; this misclassification biases the between-drone test towards no difference, so the observed effect is, if anything, conservative. Forum diagnoses were recorded with AI assistance by one recorder, verified by quotation, and half of the expert-diagnosed logs are no longer public.

Selection. The archive is self-selected, PX4-only and rich in development and test flights, and crash-rated logs differ from well-rated ones in duration and firmware era (Table 2). Results may not transfer to commercial ArduPilot fleets or to proprietary autopilots.

Measurement. The crash moment is estimated; the within-drone result did not depend on the threshold. The between-drone effect was shown mainly with the older vibration estimate, and matching used hardware, firmware, vehicle type and, in a sensitivity analysis, airframe, but not propellers, payloads or missions.

Power. Some comparisons are small (14 pilot-caused matched sets, 12 sets with the newer vibration metric, three vibration-diagnosed logs). Null results there are not evidence of no effect.

Language models. The panel cases are generated, not real incidents, and were written by the same authors as the fixed rule, so the rule's score is closer to a training score than a test score; a blind set of cases written by someone who has not seen the rule is in preparation. Only small models were tested.

The tool. The tool's checks are engineering validation on archive data, not field evaluation. Plans in the injection test were built from the flights themselves; the description search was tested on forum titles, which are shorter and more varied than an operator's report, and its symptom lists were written by the authors; the vibration ranking has not been tested prospectively; and the three-dimensional views have had no user study.

9. Conclusion

On real, public crash logs, configuration settings did not distinguish crashes, neither rules nor a model could tell whether the pilot or the aircraft was responsible, and the flight that crashed looked like the same drone's earlier flights. What the logs did reveal was which drones were rougher than their peers. Log analysis for operators is better aimed at fleet-level inspection ranking and at presenting evidence to a human investigator, and claims of automatic crash attribution should be tested with the crash window removed and against each aircraft's own history. A tool built on these results can be useful without naming causes: it can order the evidence, rank the drones that deserve inspection, keep a record that can be checked, and let a person see a flight or a plan in three dimensions.

CRediT authorship contribution statement

[**First Author**]: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft. [**Second Author**]: Investigation, Validation, Writing – review & editing. [Complete before submission.]

Declaration of competing interest

The authors develop the open-source fleet-assurance tool described in Section 6; it is not commercial. [Confirm; declare any other interests.]

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to write analysis code, draft and revise text, and assist the recording of forum diagnoses. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

Selection scripts, analysis code, the declared hypotheses with their commit history, derived features and the expert labels (without forum text), and

the source code of the tool are available at [repository URL; archived release DOI]. Raw logs are public on PX4 Flight Review and are referenced by identifier.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. [Confirm.]

References

[1] European Commission, Commission Implementing Regulation (EU) 2019/947 of 24 May 2019 on the rules and procedures for the operation of unmanned aircraft, Official Journal of the European Union, L 152, 11 June 2019 (2019).

[2] European Union Aviation Safety Agency, Easy Access Rules for Unmanned Aircraft Systems (Regulations (EU) 2019/947 and 2019/945), EASA, Cologne, revision of June 2026 (2026).

[3] Ministry of Civil Aviation, Government of India, The Drone Rules, 2021, G.S.R. 589(E), Gazette of India, 25 August 2021 (2021).

[4] Federal Aviation Administration, 14 CFR Part 107: Small Unmanned Aircraft Systems, Electronic Code of Federal Regulations, as of 21 October 2026 (2026).

[5] V. S. A. Duvvuru, B. Zhang, M. Vierhauser, A. Agrawal, LLM-agents driven automated simulation testing and analysis of small uncrewed aerial systems, in: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 385–397. doi:10.1109/ICSE55347.2025.00223.

[6] S. Silalahi, T. Ahmad, H. Studiawan, F. Breitinger, DroPTC: Sentence-level drone flight log forensics using contrastive learning and explainable AI, Forensic Science International: Digital Investigation 56 (2026) 302051. doi:10.1016/j.fsidi.2026.302051.

[7] A. Keipour, M. Mousaei, S. Scherer, ALFA: A dataset for UAV fault and anomaly detection, The International Journal of Robotics Research 40 (2–3) (2021) 515–520. doi:10.1177/0278364920966642.

[8] X. Le, B. Jin, G. Cui, X. Dai, Q. Quan, RflyMAD: A dataset for multicopter fault detection and health assessment, The International Journal of Robotics Research 44 (7) (2025) 1081–1092. doi:10.1177/02783649241305153.

[9] A. Kabaoglu, S. Sariel, UAV-SEAD: State estimation anomaly dataset for UAVs, arXiv:2602.13900 (2026).

[10] G. Wild, J. Murray, G. Baxter, Exploring civil drone accidents and incidents to help prevent potential air disasters, Aerospace 3 (3) (2016) 22. doi:10.3390/aerospace3030022.

[11] Air Accidents Investigation Branch, Annual Safety Review 2025, Tech. rep., AAIB, Department for Transport, Farnborough, UK, published 4 June 2026 (2026).

[12] S. Kaufman, S. Rosset, C. Perlich, O. Stitelman, Leakage in data mining: Formulation, detection, and avoidance, ACM Transactions on Knowledge Discovery from Data 6 (4) (2012) 15:1–15:21. doi:10.1145/2382577.2382579.

[13] S. Kapoor, A. Narayanan, Leakage and the reproducibility crisis in machine-learning-based science, Patterns 4 (9) (2023) 100804. doi:10.1016/j.patter.2023.100804.

[14] L. Meier, D. Honegger, M. Pollefeys, PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms, in: 2015 IEEE International Conference on Robotics and Automation (ICRA), 2015, pp. 6235–6240. doi:10.1109/ICRA.2015.7140074.

[15] PX4 Development Team, Flight Review log archive and pyulog ULog parser, https://review.px4.io; pyulog v1.2.4, https://github.com/PX4/pyulog; BSD-3-Clause (2026).

[16] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606. doi:10.1073/pnas.1708274114.

[17] J. Wei, H. Li, Y. Li, Y. Zhao, D. Yao, B. Lu, X. Ye, S.-K. Im, Y. Wang, X. Yang, AeroTSBoost: Temporal-statistical boosting for real-world UAV telemetry anomaly mining, arXiv:2605.25639 (2026).

[18] K. Kose, J. Wing, N. A. Kose, C. Guadarrama-Trejo, A. Sowers, A. Rasheed, A UAV testbed for diagnosing hardware vulnerabilities: Quantifying sim-to-real discrepancies in PX4 flight logs, Sensors 26 (10) (2026) 3188. doi:10.3390/s26103188.

[19] R. Puchalski, W. Giernacki, UAV fault detection methods, state-of-the-art, Drones 6 (11) (2022) 330. doi:10.3390/drones6110330.

[20] E. Khalastchi, M. Kalech, On fault detection and diagnosis in robotic systems, ACM Computing Surveys 51 (1) (2018) 9:1–9:24. doi:10.1145/3146389.

[21] E. Zibaei, R. Borth, Building causal models for finding actual causes of unmanned aerial vehicle failures, Frontiers in Robotics and AI 11 (2024) 1123762. doi:10.3389/frobt.2024.1123762.

[22] D. Wang, S. Li, G. Xiao, Y. Liu, Y. Sui, An exploratory study of autopilot software bugs in unmanned aerial vehicles, in: Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2021, pp. 20–31. doi:10.1145/3468264.3468559.

[23] E. Petritoli, F. Leccese, L. Ciani, Reliability and maintenance analysis of unmanned aerial vehicles, Sensors 18 (9) (2018) 3171. doi:10.3390/s18093171.

[24] D. A. Wiegmann, S. A. Shappell, A Human Error Approach to Aviation Accident Analysis: The Human Factors Analysis and Classification System, Ashgate, Aldershot, 2003.

[25] A. P. Tvaryanas, W. T. Thompson, S. H. Constable, Human factors in remotely piloted aircraft operations: HFACS analysis of 221 mishaps over 10 years, Aviation, Space, and Environmental Medicine 77 (7) (2006) 724–732.

[26] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, I. Mordatch, Improving factuality and reasoning in language models through multiagent debate, in: Proceedings of the 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024, pp. 11733–11763.
URL https://proceedings.mlr.press/v235/du24e.html

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90. doi:10.1145/3605764.3623985.

[28] B. Frénay, M. Verleysen, Classification in the presence of label noise: A survey, IEEE Transactions on Neural Networks and Learning Systems 25 (5) (2014) 845–869. doi:10.1109/TNNLS.2013.2292894.

[29] E. A. Stuart, Matching methods for causal inference: A review and a look forward, Statistical Science 25 (1) (2010) 1–21. doi:10.1214/09-STS313.

[30] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Communications 7 (1) (1994) 39–59. doi:10.3233/AIC-1994-7104.

[31] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (4) (2009) 333–389. doi:10.1561/1500000019.

[32] R. Stoakley, M. J. Conway, R. Pausch, Virtual reality on a WIM: Interactive worlds in miniature, in: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '95), 1995, pp. 265–272. doi:10.1145/223904.223938.

[33] J. A. Hanley, B. J. McNeil, The meaning and use of the area under a receiver operating characteristic (ROC) curve, Radiology 143 (1) (1982) 29–36. doi:10.1148/radiology.143.1.7063747.

[34] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, The Annals of Mathematical Statistics 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.

[35] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.

[36] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.
URL https://www.jstor.org/stable/4615733